joseph-branch

calculate_positions() Optimization

Overview

Integrated four optimizations applied to target runtime inefficiencies present in the function: calculate_postitions(). Modifications reduce runtime by eliminating a repeated full-scan originally used heavily for satellite navigation data lookup and preventing empty processing chunks from being used during concurrency. Mathematical computation, orbital model, and output formatting were not modified to preserve the integrity of the original program outputs. Output CSV files produced by the optimized implementation have been tested to be 100% identical to the corresponding output files produced by the original implementation.

The optimizations span four functions in rnx2db.py:

- calculate_positions()
- get_satellite_positions_process_worker()
- get_satellite_position()
- find_nearest_block()

Analysis

Bottleneck: find_nearest_block()

Before optimizing, find_nearest_block() used following logic for every call:

```
def find_nearest_block(self, prn, t):
            nav = self.nav[self.nav.index.get_level_values('sv') == prn]
            idx = (nav['MJS'] - t).abs().argsort().iat[0]
            return nav.iloc[idx]
```

The first line filters the entire navigation DataFrame by PRN for every call. Each row calls find_nearest_block(), which is scanned once per observation row meaning it is an O(N.nav) operation repeated O(N.obs) times. Original complexity can be verified to be O(N.nav * N.obs) for the lookup phase alone.

Chunk Cap:

Original code set chunk count to num_procs * 4 unconditionally:

```
chunks = num_procs * 4
obs_partition = np.array_split(self.obs, chunks)
```

Specifically for small observation files or when num_procs is large, chunks defined in calculate_positions() could exceed len(self.obs). This would cause numpy.array_split to create empty partition chunks. Each empty partition still consumed a worker slot in the process pool and produced an empty DataFrame in the results lists, adding overhead for the final pd.concat() call.

imap/starmap Migration

The original worker function accepted only a single argument(sat_list), which matched the pattern in pool.imap(). Once nav_by_prn was created to be passed alongside each chunk, imap() was no longer sufficient because it cannot unpack tuple arguments. The switch to pool.starmap() with explicit (chunk, nav_by_prn) tuples allows each worker to receive both arguments cleanly.

Optimizations

Pre-group nav data by PRN before using multiprocessing

calculate_positions(): immediately before chunk creation

Before:

```
# (no pre-grouping existed; each worker re-scanned self.nav per row)
```

After:

```
nav_by_prn = {
       prn: group
       for prn, group in self.nav.groupby(
           self.nav.index.get_level_values('sv'))
}
```

This builds a Python dictionary keyed by PRN string (e.g 'G01') where each value is the sub-DataFrame of navigation records for that satellite. groupby() performs single O(N.nav) pass over navigation data. All subsequent lookups are O(1) dict.get() calls, replacing the O(N.nav) per-row filter that previously bottlenecked the runtime.

The dict is built once in the main process before the process is created, then passed to each worker as part of the argument tuple.

Cap chunk count to prevent empty partitions

calculate_positions(): chunk count line

Before:

```
chunks = num_procs * 4
```

After:

```
chunks = min(num_procs * 4, len(self.obs))
```

The min() ensures that the number of chunks never exceeds the number of observation rows. numpy.array_split() handles uneven splits correctly, but when chunks > len(self.obs) it produces empty DataFrames. These empty chunks waste processing time and add overhead to pd.concat(). Empty chunks are eliminated entirely with a single additional function call.

Pass nav_by_prn into the worker and forward to each row

get_satellite_position_process_worker() and get_satellite_position():

Before:

```
def get_satellite_position_process_worker(self, sat_list):
       sat_list[['PosX','PosY','PosZ','ClkCorr']] = sat_list.apply(
           lambda x: self.get_satellite_position(x),
           axis=1, result_type='expand')
return sat_list
```

After:

```
def get_satellite_position_process_worker(self, sat_list, nav_by_prn):
       sat_list[['PosX','PosY','PosZ','ClkCorr']] = sat_list.apply(
           lambda x: self.get_satellite_position(x, nav_by_prn),
           axis=1, result_type='expand')
return sat_list
```

Worker processes now accept nav_by_prn as a second argument and closes over it in the lambda passed to apply(). This threads the pre-grouped dictionary through to every row, with each worker process owns a copy of the data

get_satellite_position() is updated to accept nav_by_prn=None as an optional keyword argument (defaulting to None when arg is not given). Keyword gets forwarded to get_satellite_position_std() and get_satellite_position_glonass(), which end up getting passed to find_nearest_block()

PRN lookup in find_nearest_block()

find_nearest_block():

Before:

```
def find_nearest_block(self, prn, t):
        nav = self.nav[self.nav.index.get_level_values('sv') == prn]
        idx = (nav['MJS'] - t).abs().argsort().iat[0]
        return nav.iloc[idx]
```

After:

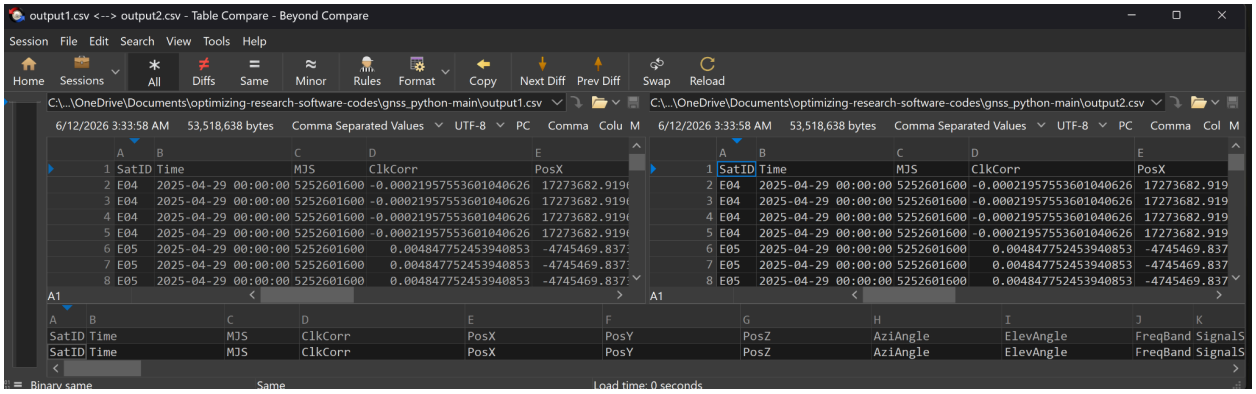
```
def find_nearest_block(self, prn, t, nav_by_prn=None):
        if nav_by_prn is not None:
            nav = nav_by_prn.get(prn)
            if nav is None:
                return None
        else:
            nav = self.nav[self.nav.index.get_level_values('sv') == prn]
        idx = (nav['MJS'] - t).abs().argsort().iat[0]
        return nav.iloc[idx]
```

When nav_by_prn is provided using the multiprocessing path, the full DataFrame filter is replaced with a single dictionary lookup. When nav_by_prn is None (meaning no multiprocessing - single-core fallback), the original behavior is preserved exactly. The dual path design guarantees that the single core fallback continues to produce the same results that it always did, and that any code that calls find_nearest_block() directly without a nav_by_prn argument continues to work without modification.

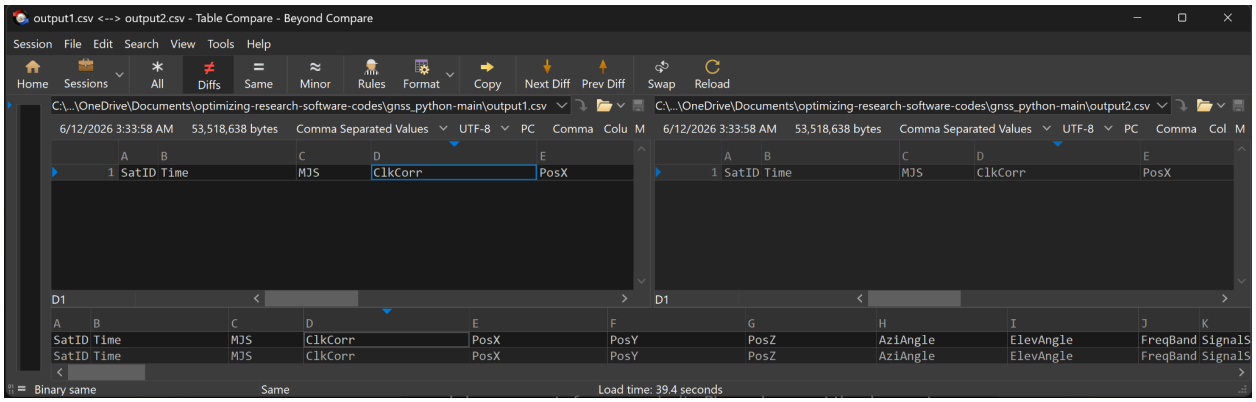
Testing

Testing with NOME1190_rnx2.25O and NOME1190.25P

Output Accuracy:



*All**



Diffs !=

Runtime Comparison:
Resulting from running original implementation against optimized implementation three times:

Initial implementation tests:

```
03:33:58 - Total runtime: 170.35 seconds (3 minutes).
03:33:58 - ====================================
```

Run 1

```
03:49:12 - Total runtime: 181.10 seconds (3 minutes).
03:49:12 - ====================================
```

Run 2

```
04:08:40 - Total runtime: 179.65 seconds (3 minutes).
04:08:40 - ====================================
```

Run 3

Optimized implementation tests:

```
03:37:43 - Total runtime: 161.95 seconds (3 minutes).
03:37:43 - ====================================
```

Run 1

```
04:02:36 - Total runtime: 159.95 seconds (3 minutes).
04:02:36 - ====================================
```

Run 2

```
04:20:57 - Total runtime: 157.85 seconds (3 minutes).
04:20:57 - ===================================
```

Run 3

Initial average: (run1 + run2 + run3) / 3 = 177.03 seconds
Optimized average: (run1 + run2 + run3) / 3 = 159.82 seconds

Small Input Test Summary:
Time saved: 177.03 - 159.82 = optimized test ran on average 17.21 seconds faster
Percentage reduction of runtime:

100 - ((159.82 / 177.03) * 100) = 9.72% runtime improvement

Future Work

- Medium and Large data sets have yet to be tested. Further testing against larger input data is still needed.
- pandas and xarray libraries are very sensitive to their version specified in requirements.txt. Further modifications to expand the support of multiple library versions would be greatly beneficial.
- Resolve the dependency errors that still occasionally occur.
- Look into optimizing mathematical models used for position calculation. Computation is still runtime expensive within the scope of calculate_positions().